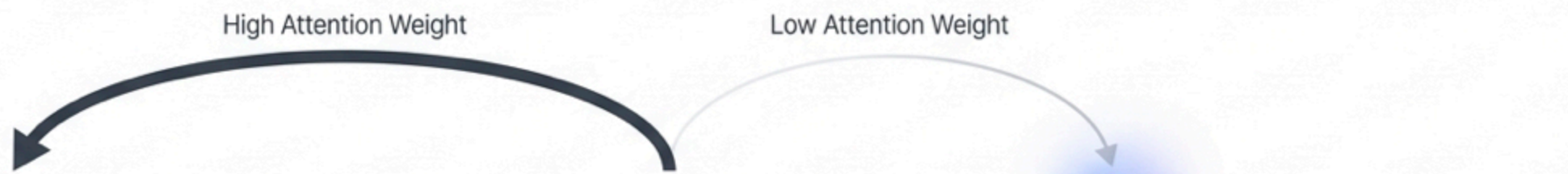
Python 人工智慧與 LLM 應用

第十章：Google Gemini API 串接與 Agent 開發

講師：Python 程式設計教學團隊

The Core Engine: Transformers and Self-Attention



The Problem: Traditional Recurrent Neural Networks (RNNs) suffer from vanishing gradients—they forget early words in long text sequences.

The 2017 Breakthrough: Transformers discarded the recurrent loop, evaluating entire text sequences simultaneously rather than sequentially.

Next-Token Prediction: An LLM is fundamentally a massive probability engine. It analyzes context weights to mathematically predict the single most likely next word.

Gemini Notebook

10.1 生成式 AI 與大型語言模型架構原理

- **Transformer & Self-Attention**：
  - **Self-Attention** (自注意力機制)：動態計算詞與上下文中其他詞的權重關聯度，克服長距離記憶退化。
  - **Next-token Prediction** (文字接龍)：模型基於概率預測下一個最可能的標記並輸出。
- **標記與計費**：
  - 以 Token 量計費，長度受限於 Context Window 窗口大小。
- **超參數控制**：
  - **Temperature** (溫度)：控制概率分佈平滑度。接近 0 輸出確定性強；接近 1 具隨機與創造性。
  - **Top-P & Top-K**：篩選候選 Token 集合範圍。

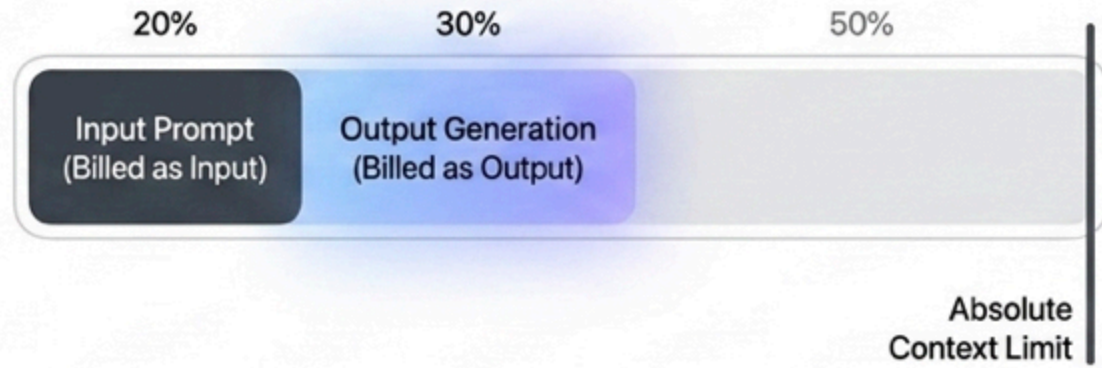
The Currency of Computation: Tokens and Context

Tokenization Process



Text is parsed into integer IDs. Rule of thumb: 1 English word ≈ 1.3 tokens.

Context Window Limits



The strict mathematical limit of what the model can process in a single request. Billed universally per 1-Million Tokens.

Gemini Notebook

Controlling Determinism: The Hyperparameter Matrix

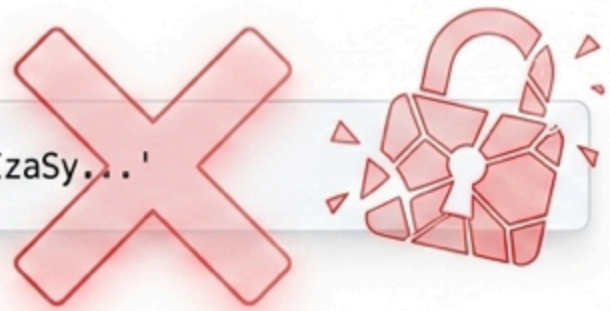
Parameter		Mechanism	Developer Use Case
Temperature		Flattens or sharpens the probability distribution of the next token.	Set to 0.0 for code generation or JSON extraction (forces stable, highly probable tokens). Set to 1.0+ for creative brainstorming.
Top-P (Nucleus)		Truncates the token candidate pool to only those whose cumulative probability reaches P.	Use to remove bizarre long-tail hallucinations while maintaining natural conversational variety.
Top-K		A hard mathematical cutoff. Only the top K most probable tokens are considered.	Use for strict vocabulary constraints and ensuring highly predictable sentence structures.

Gemini Notebook

Hello Gemini: API Initialization and Security Baseline

```
pip install google-generativeai
```

```
api_key = 'AIzaSy...'
```



Never hardcode API keys in source files. Hardcoded keys in version control are scraped and exploited within minutes.

```python
import google.generativeai as genai
import os

# Securely retrieve the key from OS environment variables
api_key = os.environ.get('GEMINI_API_KEY')
genai.configure(api_key=api_key)

# Initialize the cost-effective, high-speed model
model = genai.GenerativeModel('gemini-1.5-flash')
response = model.generate_content('Explain Python file I/O.')
print(response.text)
```

Concept Check Question (CCQ 1)

在設計一個用來進行「自動寫程式與編譯 Debug」的 AI 代理人時，你應該如何調整 Gemini API 的 `temperature` (溫度) 參數以確保程式碼穩定？

- **A.** 調高溫度至 1.0 或以上，以激發 AI 的無限創造力
- **B.** 調低溫度至 0.0 或接近 0，使模型生成最確定、最符合語法邏輯的代碼
- **C.** 關閉 Top-P 與 Top-K，只使用 Temperature=1.5
- **D.** 將溫度設為 -1.0



CCQ 1 - 答案與解析

正確答案：B. 調低溫度至 0.0 或接近 0，使模型生成最確定、最符合語法邏輯的代碼



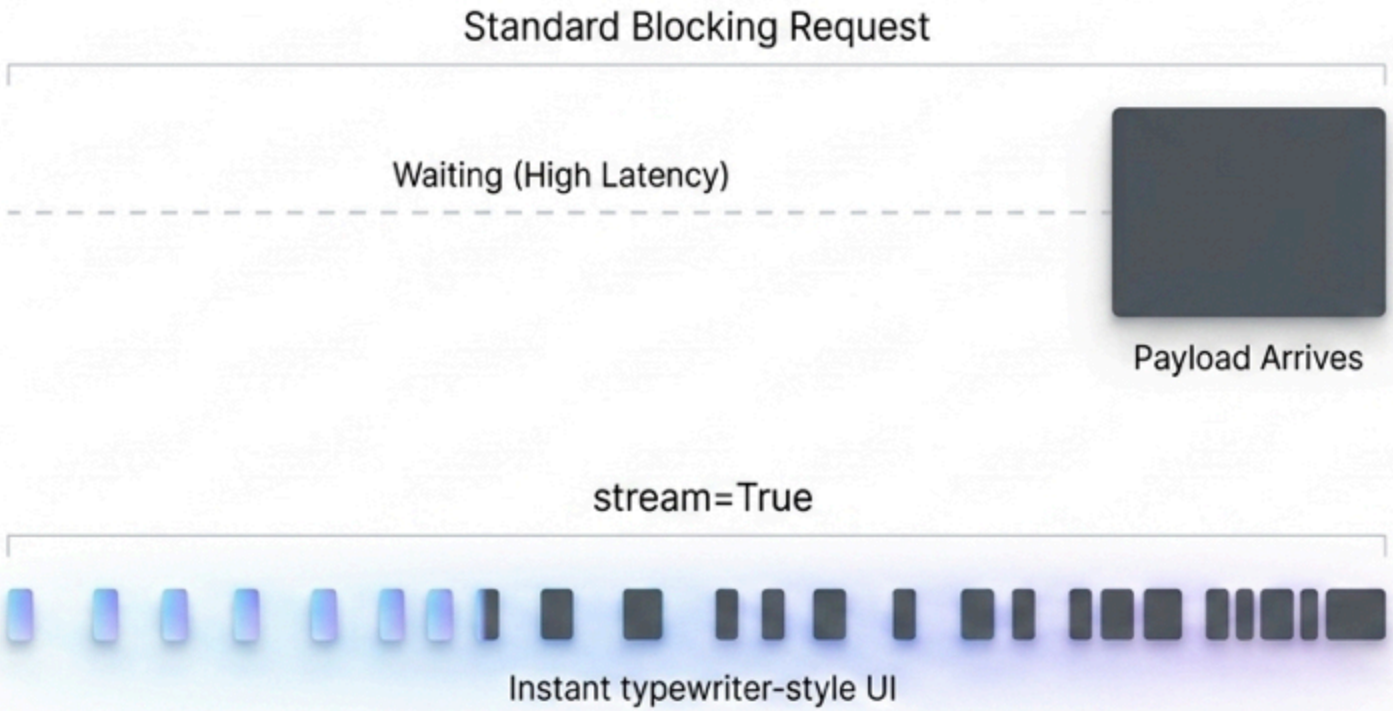
- **解析**：
  - 對於數值計算、語法編寫與邏輯推理，必須要求「高一致性」與「高可重現性」，因此需要將 Temperature 設定為 0。
  - 溫度太高會讓模型從概率較低的備選詞中抽選，造成邏輯混亂或產生程式語法幻覺，故選 B。

10.2 Google Gemini API 開發入門

- **金鑰安全保護**：
  - 嚴禁將金鑰寫死，必須存放在系統環境變數中，使用 `os.environ.get("GEMINI_API_KEY")` 來載入。
- **Google 官方 SDK**：
  - 套件名稱為 `google-generativeai`。
- **開發技巧**：
  - **串流輸出 (Streaming Response)**：
    - `generate_content_stream()`：展示打字機動態刷新效果，優化使用者體驗。
  - **多輪對話歷史 (Chat Session)**：
    - `start_chat()`：在記憶體中維護前後對話狀態。

Capability 1: Streaming Responses for Latency Mitigation

Waiting for a massive generation to complete creates the illusion that the application has frozen. Streaming solves this.



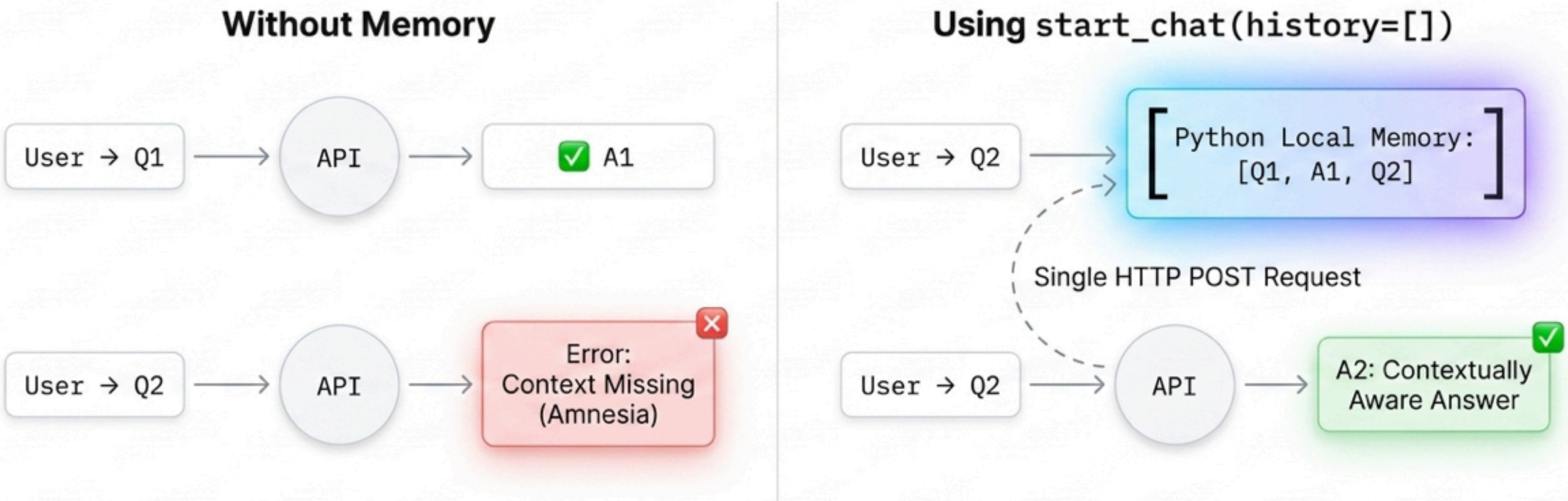

```
response_stream = model.generate_content(prompt, stream=True)

for chunk in response_stream:
    # Write directly to standard output
    sys.stdout.write(chunk.text)
    # Flush the buffer immediately
    sys.stdout.flush()
```

Gemini Notebook

Capability 2: State Management and Chat Sessions

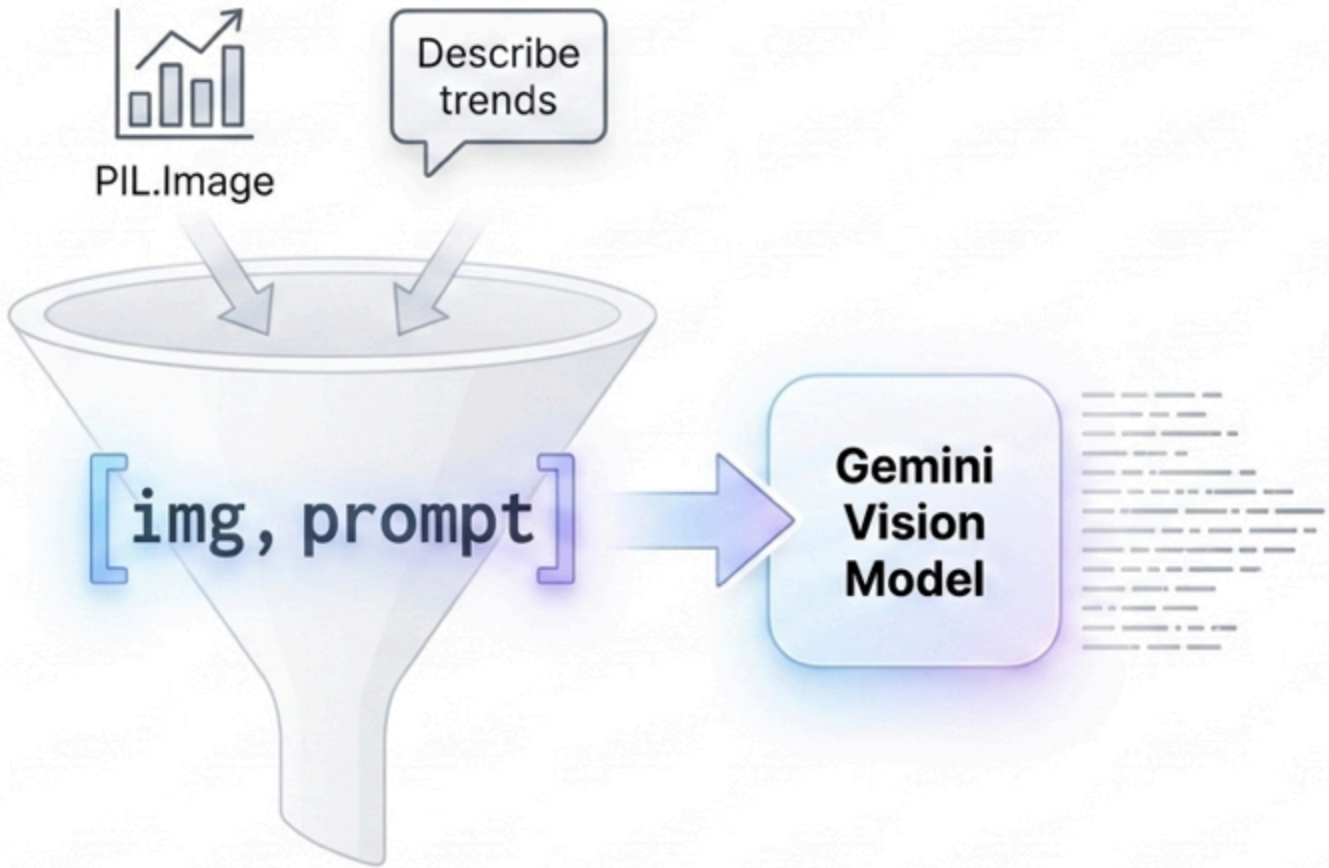
LLM endpoints are inherently stateless. The SDK manages memory locally, not via persistent WebSockets.



Gemini Notebook

Capability 3: Multimodal Vision Processing

Modern models possess native multimodal support. Pass an array of images and text directly to the API.




```
from PIL import Image

# Load the local image file
img = Image.open('sample_diagram.jpg')

# Pass the image object and prompt as a list
response = model.generate_content([
    img,
    'Describe the data trends in this chart
and provide three conclusions.'
])
print(response.text)
```

Capability 4: Forcing Structured Output (JSON Mode)

To integrate LLMs into automated data pipelines, natural language is insufficient. We must force deterministic data structures.

Here are the three books you asked for... the first one is "Python Crash Course" by Eric Matthes, which is great for beginners. The second one is "Automate the Boring Stuff with Python" by Al Sweigart, also beginner-friendly. Finally, there's "Learning Python" by Mark Lutz, which covers more detail...



response_mime_type: "application/json"

```
[
  {
    {"title": "Title",
    "author": "Author",
    "level": "Beginner"}
  }
]
```

```python
import json

# Configure generation for strict JSON compliance
model = genai.GenerativeModel(
    model_name='gemini-1.5-flash',
    generation_config={'response_mime_type': 'application/json'}
)

prompt = 'Provide 3 Python learning books. Return a JSON array.'
response = model.generate_content(prompt)

# Safely parse the guaranteed JSON string into a Python dictionary
data = json.loads(response.text)
```

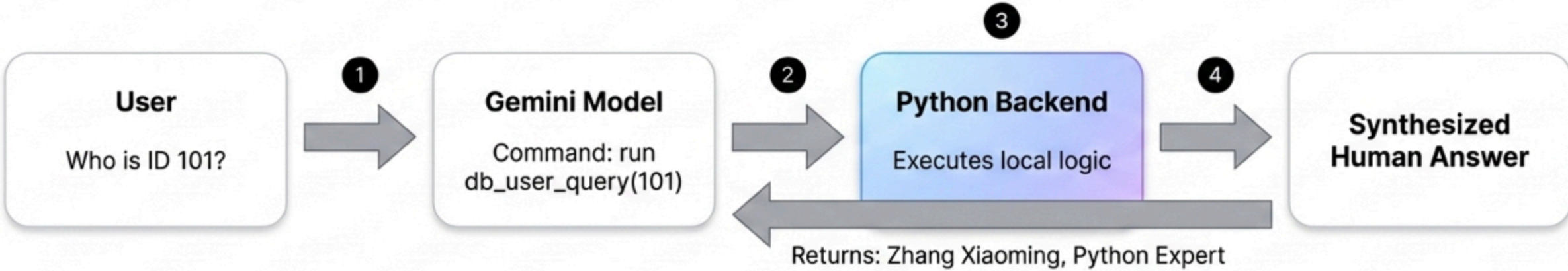
Gemini Notebook

10.3 進階開發技巧與 API 串接

- **多模態辨識 (Multimodality)**：
  - Gemini 可原生同時處理文字提示詞與 PIL 影像圖片輸入。
- **結構化 JSON 輸出模式 (JSON Mode)**：
  - 設定 `generation_config={"response_mime_type": "application/json"}`。
  - 強制模型回傳可被 `json.loads` 解析的合法格式，方便對接系統資料庫。
- **工具調用 (Function Calling)**：
  - 讓 LLM 自動識別使用者意圖，決定何時呼叫本地 Python 自訂工具函數並擷取對應參數。

Capability 5: Tool Use and Function Calling

Function calling allows the LLM to break out of its isolated sandbox and trigger local Python execution.



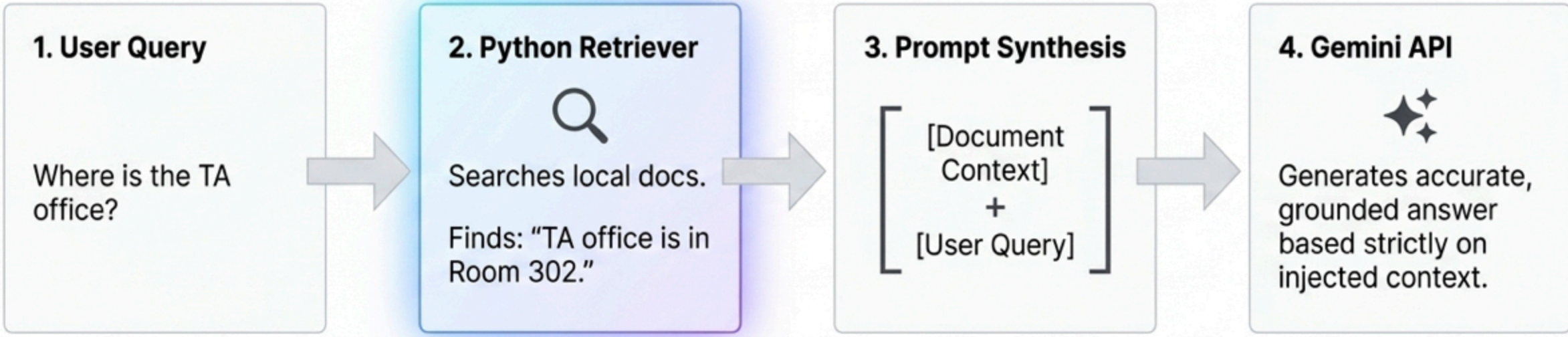

```
def db_user_query(user_id: str) -> str:
    '''Queries local database for user expertise.'''
    db = {'101': 'Zhang Xiaoming, expert in Python computing.'}
    return db.get(user_id, 'User not found.')

# Bind the Python function to the model
model = genai.GenerativeModel(
    model_name='gemini-1.5-flash',
    tools=[db_user_query]
)
chat = model.start_chat(enable_automatic_function_calling=True)
response = chat.send_message('What is the expertise of employee 101?')
```

Gemini Notebook

Capability 6: Retrieval-Augmented Generation (RAG)

Solving hallucinations by using Python as a librarian to retrieve facts before generation.



The Concept:

Do not expect the LLM to memorize your data. Treat it as a reasoning engine, not a database.

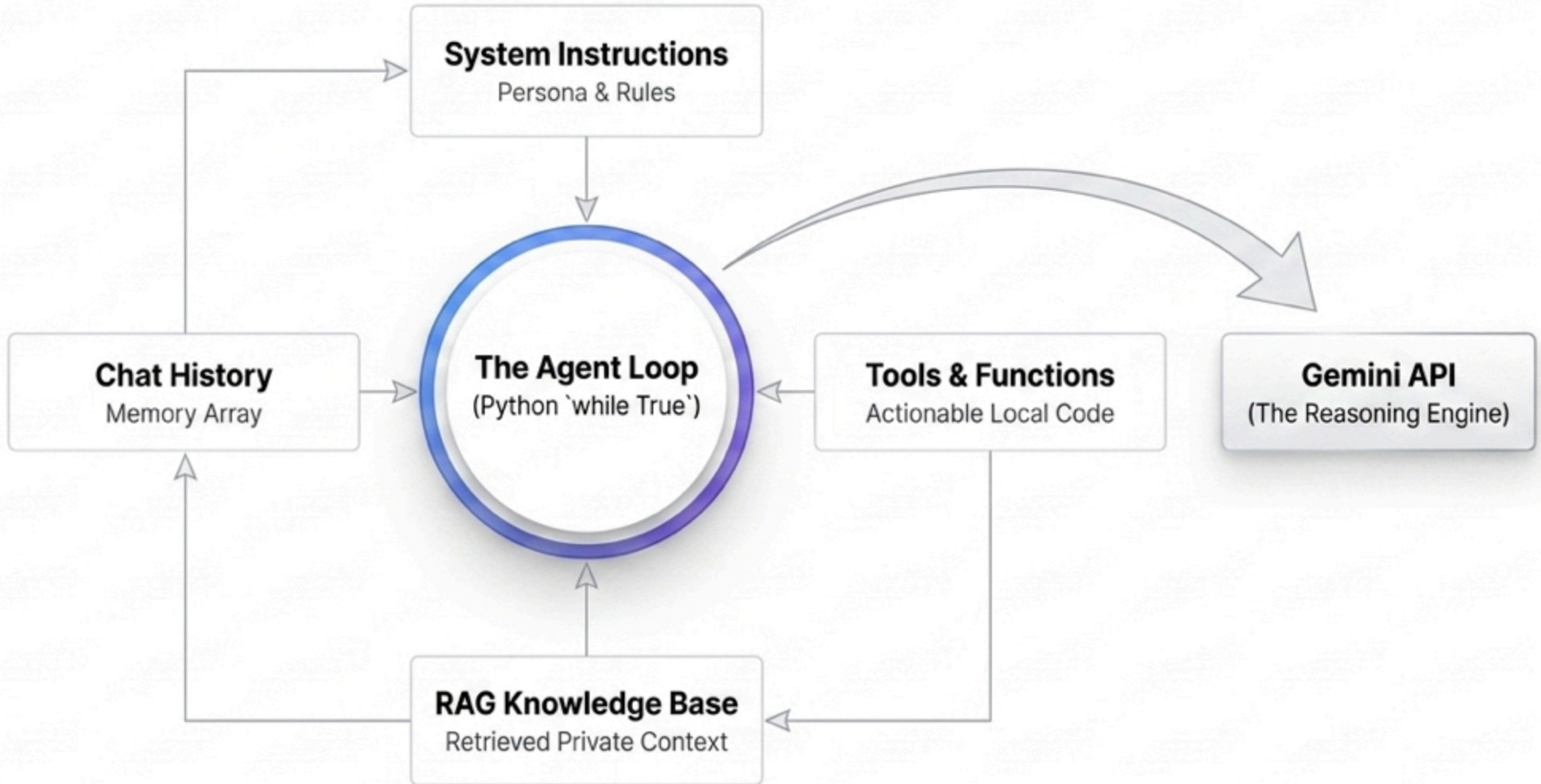
The Workflow:

Python intercepts the query, fetches localized facts, bundles them together, and asks Gemini to synthesize.

Gemini Notebook

Synthesis: Architecture of an AI Agent

An “Agent” is not magic. It is simply an LLM orchestrated by a Python event loop, augmented with modular capabilities.



Gemini Notebook

Concept Check Question (CCQ 2)

關於大型語言模型 (LLM) 的「Function Calling (工具調用)」機制，下列敘述何者正確？

- **A.** 該機制允許 LLM 繞過作業系統權限，在本地下載並執行 Python 原始碼
- **B.** LLM 不直接執行該函數；僅負責識別使用者意圖，並輸出包含「函數名稱與引數數值」的指令，由開發者本地執行
- **C.** Function Calling 是一種用於微調 (Fine-Tuning) 模型參數的演算法
- **D.** 它會將模型本身的硬體推論速度提升 100 倍



CCQ 2 - 答案與解析

正確答案：B. LLM 不直接執行該函數；僅負責識別使用者意圖，並輸出包含「函數名稱與引數數值」的指令，由開發者本地執行



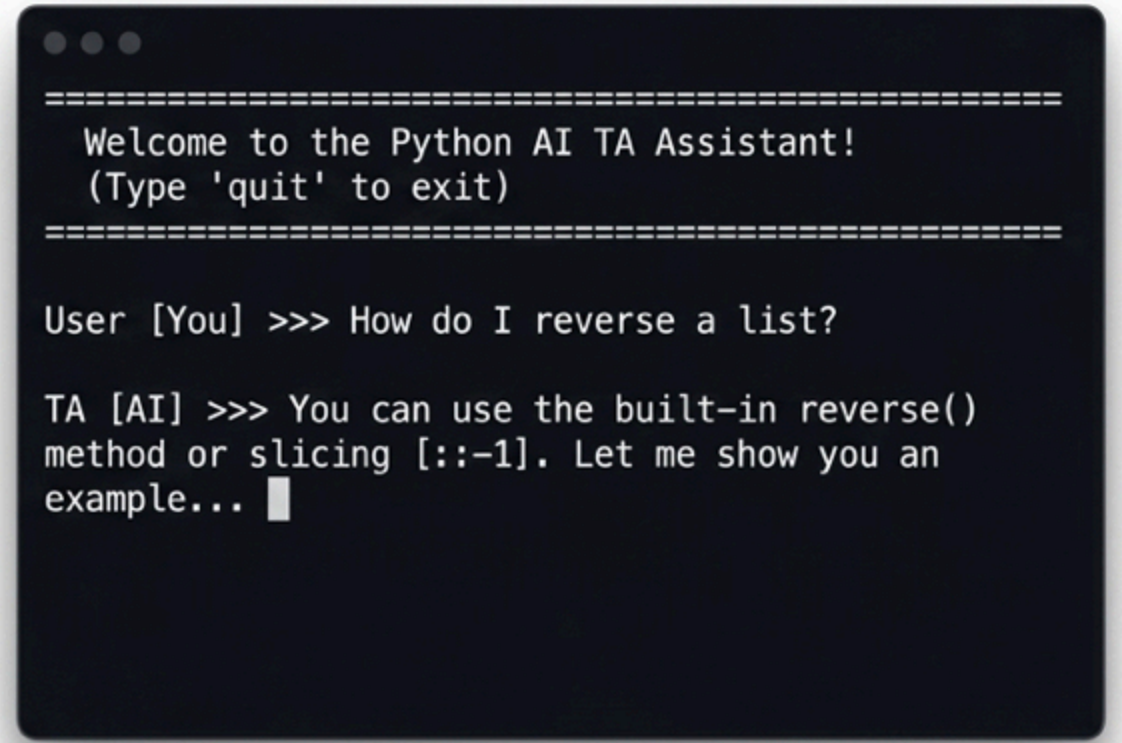
- **解析**：
  - 模型本身無法與外部伺服器或本機環境直接發生實體互動。
  - 機制本質是：模型輸出結構化指令指引，要求開發者的本地系統執行該函數並將返回值發送給模型進行下一輪分析，故選 B。

10.4 檢索增強生成 (Retrieval-Augmented Generation)

- **為什麼需要 RAG**：
  - 解決模型幻覺 (Hallucination)。
  - 提供時效性與內部私人機密文件的知識參考。
- **RAG 工作流架構**：
  1. **檢索 (Retrieve)**：利用相似度計算（如向量/關鍵字交集）在資料庫中找出最相關的文字段落。
  2. **增強 (Augment)**：將檢索文字併入 System / Prompt 上下文中。
  3. **生成 (Generate)**：要求 LLM 僅基於給定的上下文回答使用者問題。

Capstone: The Terminal AI Assistant

Combining System Prompts, Chat Sessions, and Streaming to build a fully functional, localized AI tutor.




```python
model = genai.GenerativeModel(
    model_name='gemini-1.5-flash',
    system_instruction='You are a CS teaching assistant.'
)
chat = model.start_chat(history=[])

while True:
    user_input = input('\nUser [You] >>> ').strip()
    if user_input.lower() == 'quit': break

    # Send user input and stream response live
    response_stream = chat.send_message(user_input, stream=True)
    for chunk in response_stream:
        sys.stdout.write(chunk.text)
        sys.stdout.flush()
```

Gemini Notebook

Concept Check Question (CCQ 3)

在實作 RAG (檢索增強生成) 系統時，將檢索到的私人參考文件作為背景 Context 放入 Prompt 中，主要是為了解決什麼痛點？

- **A.** 網路傳輸頻寬不足的問題
- **B.** 解決模型因為訓練資料截止或缺乏私人知識而產生的幻覺 (Hallucination) 問題，以提供有憑有據的解答
- **C.** 提升本地顯示卡的硬體推論算力
- **D.** 自動將輸入的程式碼進行最佳化編譯



CCQ 3 - 答案與解析

正確答案：B. 解決模型因為訓練資料截止或缺乏私人知識而產生的幻覺 (Hallucination) 問題，以提供有憑有據的解答



- **解析**：
  - 將檢索到的相關內容以上下文形式提供給模型，相當於「開卷考試」，模型能從中提取資料回答，免除隨意亂猜導致的幻覺，並可提供準確的出處來源，故選 B。

10.5 本章綜合實作專題

- **終端機 AI 課程助教 Agent**：
  - 設定 `system_instruction`，限定其角色語氣為正體中文的「程式設計助教」。
  - 限制：禁止直接給出正確解答代碼，應逐步給予提示並引導學生除錯思考。
  - 使用多輪對話歷史追蹤學生提交的錯誤。

Architectural Truths of AI Development



Theory & Mechanics

LLMs are probability engines executing next-token prediction via Self-Attention.

They process integer tokens, not human words.

They are constrained by strict Context Windows.

Temperatures near 0.0 guarantee deterministic outputs.



The API is Stateless

The generative model remembers absolutely nothing between queries.

Stateful Chat Sessions are maintained entirely in local Python memory.

Every chat turn bundles the full history into a brand new HTTP POST.



Python is the Orchestrator

The LLM provides raw reasoning; Python provides the infrastructure.

True application power unlocks when Python intercepts outputs to parse JSON, execute local Functions, or fetch RAG context.

Gemini Notebook